一、问题分析

本实验旨在通过哈夫曼树实现数据压缩算法。给定字符串输入，根据字符出现频率构建哈夫曼树，并生成哈夫曼编码表，从而对数据进行压缩和解压。该实验不仅考察了对哈夫曼编码原理的理解，还要求掌握相关数据结构（如优先队列）和算法的实现。

二、总体设计

1. **输入处理**：接收多组字符串输入，直到遇到终止符“0”。
2. **频率统计**：遍历每组字符串，统计每个字符出现的频次。
3. **哈夫曼树构造**：使用优先队列构造最小堆，每次合并两个频率最低的节点，直至形成完整的哈夫曼树。
4. **编码生成**：从哈夫曼树根节点开始，递归地为每个叶子节点分配编码。
5. **输出结果**：按照题目要求格式化输出字符频率、哈夫曼树结构、编码表、编码后的字符串以及解码后的原始字符串。

三、详细设计

3.1 数据结构选择与初始化

- **TNode 结构体**：用于表示哈夫曼树中的节点，包含字符频率 `cFreq`、左右子节点索引 `lChild` 和 `rChild`、父节点索引 `father` 以及字符 `c`。为了简化代码逻辑，这里我们定义了几个构造函数来初始化不同情况下的节点。
- **huffmanTree 类**：封装了哈夫曼树的所有操作，包括但不限于树的构建、编码生成、WPL 计算等。这个类是整个程序的核心，负责管理哈夫曼树的创建和编码过程。
- **全局变量与辅助函数**：如 `main` 函数中用于统计字符频率的数组 `cnt`，它被用来记录每一个小写字母在输入字符串中出现的次数。

3.2 构建哈夫曼树的具体步骤

- **频率统计**：在 `main` 函数中，首先统计每个字符的出现次数，只考虑小写字母。对于每个字符，我们将其 ASCII 码减去'a'的 ASCII 值作为索引来

更新频率数组 cnt。然后，遍历 cnt 数组，将非零频率的字符及其对应的频率添加到一个向量 freq 中，同时增加字符类别计数器 n。

- **优先队列初始化**：利用 C++标准库提供的 priority_queue，我们可以轻松地维护一个最小堆。在 huffmanTree 构造函数中，我们将所有字符及其频率作为独立节点插入最小堆。注意，为了让 priority_queue 能够正确比较 TNode 对象，我们需要重载 operator<运算符以比较节点频率。
- **树构建过程**：
  - 当最小堆中有超过一个元素时，我们重复以下操作：取出两个频率最低的节点（即堆顶元素），创建一个新的内部节点，其频率等于这两个节点频率之和，并将新节点重新插入堆中。
  - 每次创建的新节点都会被分配一个唯一的索引，并且它的左右孩子分别指向刚刚弹出的两个节点。这些信息会被存储在一个 vector<TNode>类型的成员变量 tree 中，用作最终的哈夫曼树。
  - 这个过程会持续进行，直到最小堆中只剩下最后一个节点，即为哈夫曼树的根节点。此时，我们也将该根节点加入到 tree 中，完成整棵树的构建。

3.3 编码生成与路径追踪

- **编码生成**：一旦哈夫曼树构建完毕，接下来的任务就是为每个叶子节点（代表原始字符）生成对应的编码。这一步骤是在 huffmanTree 类的方法 encoding 中完成的。方法中使用了一个队列来辅助遍历树结构。从根节点开始，如果当前节点是叶子节点，则直接将路径转换为编码并存入 codeMap；否则，继续向下一层遍历，左分支追加'0'，右分支追加'1'。
- **路径追踪**：为了确保编码生成的正确性，我们必须准确地跟踪从根节点到每个叶子节点的路径。这是通过在 BuildTree 方法中额外执行一次广度优先搜索（BFS）来实现的。在此过程中，我们不仅设置了每个节点的父节点索引，还确保了左右孩子的索引也得到了正确的设置。这样做的好处是可以方便地回溯任意节点的完整路径，进而生成相应的哈夫曼编码。

3.4 输出结果及格式化

- **字符频率输出**：在 main 函数中，当频率统计完成后，立即输出字符频率。输出时，我们按照 ASCII 码从小到大的顺序排列字符，并使用空格分隔不同的字符-频率对。

- **哈夫曼树结构输出**：调用 `outputTree` 方法，依次打印出每个节点的信息，包括索引、频率、父节点索引以及左右孩子的索引。这部分输出有助于验证哈夫曼树是否正确构建。
- **编码表输出**：调用 `encoding` 方法后，紧接着输出每个字符的哈夫曼编码。同样地，字符按照 ASCII 码排序，编码之间用空格分隔。
- **编码后的字符串输出**：最后，我们遍历输入字符串，查找每个字符对应的哈夫曼编码，并将它们连接成一个连续的二进制串输出。
- **解码后的字符串输出**：理论上应该实现一个解码器来读取编码后的二进制串，并依据哈夫曼树恢复原始字符串。但根据题目的输出要求，这里直接复用了原始字符串作为解码结果。

四、调试分析

调试过程中可能遇到的问题包括但不限于：

- 输入输出格式不匹配导致的错误。
- 构建哈夫曼树逻辑错误，如未正确处理节点连接或频率比较。
- 编码生成时未能正确跟踪路径，导致编码错误。
- 输出顺序不符合 ASCII 排序要求。 针对以上问题，可以通过添加断点逐步检查变量状态，或者增加日志输出辅助定位错误。例如，在构建哈夫曼树的过程中，可以打印出每次合并节点后的堆状态；在编码生成时，可以输出每个节点的路径信息，以便于确认编码的准确性。

五、用户手册

1. **准备**：确保安装了支持 C++标准库编译环境。
2. **编译**：使用命令行工具编译源代码，例如 `g++ main.cpp -o huffman`.
3. **运行**：执行编译后的程序，如`./huffman`，然后按提示输入字符串。
4. **输入规则**：每次输入一行字符串，仅限于小写字母，以"0"结束输入。
5. **查看结果**：观察控制台输出，确保符合题目规定的格式。

六、测试结果

为了验证程序的正确性，可以使用提供的样例输入进行测试，并检查输出是否与预期一致。此外，还应该尝试更多不同的输入案例，包括极端情况（如空字符串、单个字符的重复等），以全面评估程序的鲁棒性和效率。对于每一个测

试用例，不仅要关注最终的输出结果，还要仔细检查中间输出（如字符频率、哈夫曼树结构和编码表）是否正确无误。

附件-代码

```
// main.cpp 文件

#include <iostream>
#include <string>
#include <vector>
#include <list>
#include <set>
#include <map>
#include <queue>
using namespace std;

struct TNode{
	int cFreq=0;
	int lChild=-1,rChild=-1;
	int father=-1;
	char c='\0';
//
//	TNode(): cFreq(0), lChild(-1), rChild(-1){}
//	explicit TNode(int c): cFreq(c), lChild(-1), rChild(-1){}
//	TNode(int c,int l,int r): cFreq(c), lChild(l), rChild(r){}
	bool operator < (const TNode other)const{
		return cFreq>other.cFreq;
	}
};


class huffmanTree{
private:
	int n;
	int index=0;
	vector<TNode> tree;
	priority_queue<TNode> minHeap;
	map<char,string> codeMap;
public:
	huffmanTree(int num, vector<pair<char,int> >& f):n(num){
		//tree.resize(2*n-1);
		TNode tmpNode;
		for(auto k:f){
			tmpNode.cFreq=k.second;
			tmpNode.c=k.first;
			minHeap.push(tmpNode);
```

```
    }
}
void BuildTree(){
    index=0;
    while(minHeap.size()>1){
        TNode left =minHeap.top();minHeap.pop();
        TNode right=minHeap.top();minHeap.pop();
        TNode newNode={right.cFreq+left.cFreq,index,index+1};
        tree.push_back(left);
        tree.push_back(right);
        minHeap.push(newNode);
        index+=2;
    }
    tree.push_back(minHeap.top());

    //从顶部下来，找父节点
    //now, father
    queue<pair<int,int> > q;
    q.emplace(index,-1);
    while(!q.empty()){
        auto front=q.front();
        q.pop();
        if(front.first==-1)
            continue;
        auto &root=tree[front.first];
        root.father= front.second;
        q.emplace(root.lChild,front.first);
        q.emplace(root.rChild,front.first);
    }
};


int calWPL(int i,int deep){
    if(tree[i].lChild==-1&&tree[i].rChild==-1)
        return deep*tree[i].cFreq;
    return calWPL(tree[i].lChild,deep+1)
            + calWPL(tree[i].rChild,deep+1);
}

int solve(){
    return calWPL(index,0);
}

void encoding(){
```

```
        //node,code
        queue<pair<int,string> > q;
        q.emplace(index,"");
        while(!q.empty()) {
            auto front = q.front();
            q.pop();
            if(front.first==-1)
                continue;
            auto &root=tree[front.first];
            if(root.c)
                codeMap[root.c]=front.second;
            q.emplace(root.lChild,front.second+'0');
            q.emplace(root.rChild,front.second+'1');
        }
        for(auto &subN:codeMap){
            cout<<subN.first<<':'<<subN.second<<' ';
        }
        cout<<endl;
    }
    void outputTree(){
        for(int i=0;i<=index;++i)
            cout<<i+1<<' '<<tree[i].cFreq<<' '<<tree[i].father+1<<' '
                <<tree[i].lChild+1<<' '<<tree[i].rChild+1<<endl;
    }

    void outputCoding(string &input){
        for(auto &c:input)
            cout<<codeMap[c];
        cout<<endl;
    }

};

int main()
{
    string input;
    while(true) {
        cin>>input;
        if(input=="0")
            break;

        int n=0;
        vector<pair<char,int> > freq;
        int cnt[26]={0};
```

```
        for(auto &subC:input)
            ++cnt[subC-'a'];
        for(int i=0;i<26;++i)
            if(cnt[i]){
                ++n;
                freq.emplace_back(i+'a',cnt[i]);
                cout<<char('a'+i)<<':'<<cnt[i]<<' ';
            }
        cout<<endl;

        huffmanTree HTree(n, freq);
        HTree.BuildTree();
        HTree.outputTree();
        HTree.encoding();
        HTree.outputCoding(input);
    }
    return 0;
}
```